Mobile testing on real devices: interview questions, answered with this project

Questions about mobile web and native app testing on real devices and device clouds (BrowserStack, Appium, Selenium, emulation versus real devices) that come up in SDET and QA lead interviews. Each answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/mobile-real-device-testing.

Versions: Selenium 4.51, Appium Java client 11, TestNG 7.12, Java 21, BrowserStack Automate and App Automate, iPhone 16 (iOS 18), Samsung Galaxy S24 (Android 14).

Contents

1. Real devices, emulators and device clouds (Q1-Q12)
2. Testing mobile web (Q13-Q26)
3. Native iOS apps with Appium (Q27-Q38)
4. Cost, CI, debugging and scenarios (Q39-Q52)

1. Real devices, emulators and device clouds

Q1. Emulator, simulator, real device: what is the difference?

An Android emulator runs the real OS on virtual hardware; an iOS simulator runs the app compiled for the Mac, not the real iOS. Browser mobile emulation only changes viewport, pixel ratio, touch flag and user agent in a desktop browser. A real device has the real browser engine, keyboard, gestures, performance, sensors and OS behaviour.

In the project: Chrome mobile emulation for development and CI; real iPhone and Android phones for proof.

Q2. What does emulation miss?

The on-screen keyboard and how it resizes the page, real touch handling, the real Safari engine on iOS (every iOS browser uses WebKit), device fonts, performance, memory, OS dialogs and permissions, and network conditions.

In the project: the checkout tap failed only on the real Android phone, with the keyboard open.

Q3. Why not test only on real devices?

Cost and speed. Real devices are slower to start, limited in number and paid per minute. Run most checks on emulation or desktop browsers, and the device-specific ones (plus smoke tests) on real devices.

In the project: 13 tests in 8.5 s on emulation versus about 1.5 to 2 minutes per real phone.

Q4. What is a device cloud?

A service hosting real phones and tablets that tests reach over the WebDriver protocol: BrowserStack, Sauce Labs, LambdaTest, AWS Device Farm. You choose device and OS per session; video, logs and screenshots are recorded.

In the project: BrowserStack Automate (browsers) and App Automate (native apps).

Q5. How do you choose the devices to test on?

From real usage: analytics of the product's visitors (devices, OS versions, browsers), plus the newest and the oldest supported OS. A small matrix that covers most users beats many devices nobody uses.

In the project: one current iPhone and one current Samsung, the two dominant engines (WebKit, Chromium).

Q6. Why does iOS Safari matter so much for mobile web?

On iOS every browser, including Chrome, uses Apple's WebKit engine. Bugs specific to WebKit appear for all iOS users, and desktop Chrome or Android Chrome will not show them.

In the project: the web suite runs on real iOS Safari.

Q7. How do real devices reach a site that is not on the internet?

Through a tunnel: BrowserStack Local (or Sauce Connect) runs on the test machine and forwards the device's requests to local or internal addresses. The device opens a special host name (`bs-local.com`) that routes through it.

In the project: `Tunnel` starts BrowserStack Local; the base URL is `bs-local.com:4330` on BrowserStack.

Q8. Why `bs-local.com` instead of `localhost`?

On a device, `localhost` means the device itself; iOS Safari does not send it through the tunnel at all. `bs-local.com` is BrowserStack's name for "the machine running the tunnel".

In the project: `Target.baseUrl`.

Q9. What is a local identifier for the tunnel?

A name tying sessions to one tunnel, so parallel runs (or two developers) do not use each other's tunnels.

In the project: a random `run-<uuid>` per run.

Q10. How do you pass credentials to a device cloud safely?

From environment variables or CI secrets, never in code or files, never printed in logs.

In the project: `BrowserStack.username()` reads the environment and fails with a clear message if missing.

Q11. What does BrowserStack record for a session?

Video, screenshots, Selenium or Appium logs, network and console logs, device logs. They show what happened even when the test's own evidence is not enough.

In the project: the session logs showed the click timing during the Android diagnosis.

Q12. How does the cloud know whether a test passed?

It does not: it only sees commands. You report the status, for BrowserStack with a `browserstack_executor` script (`setSessionStatus`).

In the project: `WebDrivers.reportStatus` and `AppTestBase.endSession`.

2. Testing mobile web

Q13. What do you check first for a mobile website?

The viewport meta tag, layout at phone width (no sideways scrolling, readable text, one column where intended), navigation that works without hover, touch target size, form keyboards, and the main journeys.

In the project: `MobileLayoutTest`, `TouchAndInputTest`.

Q14. What happens without the viewport meta tag?

The phone lays the page out about 980 px wide and shrinks it to fit: tiny text, pinch-zoom needed. Responsive CSS breakpoints never apply.

In the project: `viewportIsSetForPhones`.

Q15. How do you test for horizontal scrolling?

Compare `document.documentElement.scrollWidth` with `clientWidth`; any difference is sideways scrolling.

In the project: `noPageScrollsSideways` on four pages.

Q16. How big should touch targets be?

Apple recommends 44 × 44 points, Android 48 × 48 dp; WCAG 2.5.5 (AAA) asks for 44 × 44 CSS px and 2.5.8 (AA) for at least 24 × 24. Measure with `getBoundingClientRect`.

In the project: `touchTargetsAreBigEnough`.

Q17. Why test input types and attributes?

On a phone, `type="email"` brings the @ keyboard, `type="tel"` and `inputmode="numeric"` the number pad, and `autocomplete` lets the browser fill saved details. Wrong attributes make forms slow and error-prone on phones.

In the project: `checkoutFieldsChooseTheRightKeyboard`.

Q18. What is the iOS zoom-on-focus problem?

iOS Safari zooms into an input whose font size is below 16 px when it gets focus, and does not zoom back. Inputs at 16 px or more avoid it.

In the project: `inputsAreLargeEnoughToAvoidZoomOnFocus`.

Q19. How do you prove a test really ran on a phone?

Assert device facts: `window.innerWidth` in the phone range and `navigator.maxTouchPoints` above 0. A misconfigured run on a desktop browser then fails instead of passing.

In the project: `browserIsAPhone`.

Q20. How do you test a hamburger menu?

The inline menu is hidden at phone width, the button reports `aria-expanded="false"`; a tap opens the menu, `aria-expanded` becomes true, and a link navigates.

In the project: `menuOpensFromTheMenuButton`.

Q21. What is Chrome mobile emulation good for?

Fast feedback on layout and logic at phone sizes, with touch events and a mobile user agent, in any CI. It is not a real browser on a phone.

In the project: `WebDrivers.localPhone`.

Q22. Why reset state with localStorage instead of a new session?

A new real-device session costs start-up time and minutes. The shop keeps its cart in localStorage, so clearing it gives each test a clean shop in milliseconds.

In the project: `WebTestBase.freshShop`.

Q23. How do you wait after a navigation?

Wait for something only the new page has, or for the expected text, never "read whatever is there now".

In the project: `PocketShop.awaitHeading`.

Q24. What did the Android phone show that emulation did not?

With the on-screen keyboard open, the automated tap on the checkout button landed on `<html>`, so the form was never submitted. Closing the keyboard and waiting for the layout to settle fixed it.

In the project: `PocketShop.tapAfterTyping`.

Q25. How did you find that cause?

A probe in a single-test run: listeners recorded what received the click and whether a submit fired, plus the focused element, viewport height and button position. My first guess, a timing race, turned out to be wrong.

In the project: the README's "What the real devices found".

Q26. Can you use Playwright for real iOS devices?

Playwright drives its own WebKit build, not Safari on a real iPhone; device clouds offer real iOS Safari through WebDriver (Selenium or Appium). Playwright's WebKit is a good, fast approximation, not a replacement.

In the project: Selenium for real devices.

3. Native iOS apps with Appium

Q27. How does Appium automate iOS?

Through the XCUITest driver, which uses Apple's XCUITest framework (WebDriverAgent) on the device. Tests talk WebDriver to Appium; Appium talks to the device.

In the project: `IOSDriver` with `XCUITestOptions`.

Q28. Which iOS locators do you use?

Accessibility ids first (stable, set by developers, also good for VoiceOver), then iOS class chains or predicate strings, XPath last (slow on iOS).

In the project: accessibility ids and class chains, no XPath.

Q29. How do you find locators without a Mac?

Appium Inspector on a cloud device, or, cheaper, one session that saves `getPageSource()` of each screen to files and read them offline.

In the project: a 26-second discovery session saved the element tree of each tab.

Q30. Why can't any IPA run on a real iPhone?

iOS only runs apps signed with a certificate and a provisioning profile that allow the device. Device clouds re-sign uploads, which needs a development or ad-hoc (or enterprise) build with a profile inside.

In the project: the Sauce Labs demo app was rejected; BrowserStack's sample app was accepted.

Q31. How do you upload an app to BrowserStack?

The REST API (`/app-automate/upload`) with a file or URL, optionally a `custom_id`, which tests then use as the `app` capability so the newest upload with that id is used.

In the project: the README's upload command, custom id `bstack-sample-ios`.

Q32. How do you reset a native app between tests in one session?

`terminateApp` and `activateApp` by bundle id; for a full data reset, reinstall or use the app's own reset.

In the project: `AppTestBase.restartApp`.

Q33. How do you handle native alerts?

Through the WebDriver alert API (`switchTo().alert()` , with a wait for `alertIsPresent`), or `mobile: alert` commands.

In the project: `alertOpensAndCloses`.

Q34. How do you test web content inside a native app?

Either through the native accessibility tree (elements inside the web view), or by switching to the WEBVIEW context when the web view is inspectable. Check structure when the content is outside your control.

In the project: `webViewTabShowsWebContent` checks for links inside the web view.

Q35. How do you know a tab is selected?

The tab bar button reports `value="1"` when selected (seen in the element tree).

In the project: `startsOnTheUiElementsTab`.

Q36. What do you keep when a native test fails?

A screenshot and the element tree at that moment, so a locator can be fixed without spending another device run.

In the project: `AppTestBase.keepEvidence`.

Q37. Native app testing versus mobile web testing?

Native: installation, signing, permissions, gestures, OS integration, offline behaviour, upgrades. Mobile web: browsers, layout, viewport, forms and keyboards, caching. Many products need both.

In the project: both, in one repository and one framework.

Q38. How would you test the same app on Android and iOS?

Shared test scenarios and data, platform-specific page objects (or annotations per platform), the same app version on both, and a device matrix per platform.

In the project: my appium-framework covers Android; this one shows iOS on real devices.

4. Cost, CI, debugging and scenarios

Q39. How do you control the cost of a device cloud?

Develop on emulation, one session per suite, single-test runs while fixing, an idle timeout, discovery and probes instead of trial and error, and real devices in CI only on demand or on a schedule.

In the project: about 9.5 Automate and 1.3 App Automate minutes for everything.

Q40. Why are real devices manual in CI here?

A push must not spend trial minutes. A team with a paid plan would run them nightly or on release branches.

In the project: `workflow_dispatch` with a choice of devices.

Q41. What is an idle timeout, and why lower it?

How long a session may wait for the next command before the cloud ends it. A crashed test run otherwise keeps the device (and the meter) running for minutes.

In the project: 120 seconds instead of the default 300.

Q42. A test passes locally and fails on a real device. What do you do?

Look at the device's screenshot and video, read the cloud's command log, reproduce with a single-test run, and add a probe for the suspected cause instead of guessing changes.

In the project: the Android checkout diagnosis.

Q43. Is a device-specific workaround in the test acceptable?

When it does what a real user does and the reason is documented, yes. A workaround that hides a real product bug is not. Decide by asking: would a user hit this?

In the project: users close the keyboard or the tap works for them; the automated tap missed.

Q44. How do you run real-device tests in parallel?

Use the plan's parallel sessions: one session per thread (TestNG parallel classes or a matrix in CI), with unique session names and one tunnel shared through a local identifier.

In the project: the free plan allows 5; the suite runs one session per device.

Q45. Which OS version will you get?

You request a major version; the cloud may give different minor versions (18.5 one day, 18.6 the next). Record it with the results.

In the project: the results table records the version per run.

Q46. How do you test a staging app behind a firewall on real devices?

With the cloud's tunnel running inside the network (BrowserStack Local, Sauce Connect).

In the project: PocketShop runs on the test machine, reached only through the tunnel.

Q47. What makes a mobile journey test flaky on real devices?

Keyboards covering or moving elements, animations, slower page loads over the tunnel, OS pop-ups, and taps on elements still moving. Waits on conditions and settled layouts fix most of them.

In the project: `tapAfterTyping` waits until the button stops moving.

Q48. How would you test performance on a real phone?

Measure page load and interaction timing in the real browser (Navigation Timing API), with network throttling profiles, on the low-end devices your users have.

In the project: out of scope; listed in the design document.

Q49. How do you report real-device results to a team?

Per device: OS, browser or app, tests, result, time, and links to the cloud's sessions; plus anything found only on a real device and how it was fixed.

In the project: the results image in the README.

Q50. What is the role of a QA lead in a device strategy?

Choose the device matrix from data, decide what runs where (emulation, real devices, manual exploratory sessions), set the budget and schedule, and make sure failures are diagnosed rather than retried.

In the project: the minute budget and the decisions table.

Q51. Would you still do manual testing on real devices?

Yes: exploratory sessions on real phones find what scripts do not (gestures, readability, odd states). Automation keeps the known journeys working.

In the project: BrowserStack App Live or Live would be the tool for that.

Q52. What was the most useful lesson?

That the first explanation of a real-device failure can be wrong: the screenshot suggested a timing race, the probe proved the tap missed the button. Measure before changing.

In the project: the README and design document tell both steps.